SQL-DDL aus dem ER-Modell ableiten

Joern Ploennigs · AI4SC



> The problem with SQL is that it pretends to implement the relational model, but doesn't.
>
> — David McGoveran

SQL-DDL aus dem ER-Modell ableiten

In `12c_Datenbanken_Entwurf` haben wir das ER-Modell als fachliche Spezifikation entwickelt. In dieser Sitzung geht es nicht darum, `CREATE TABLE` auswendig zu lernen, sondern zu prüfen, ob von KI erzeugtes SQL diese Spezifikation korrekt implementiert.

Das fachliche Ziel ist daher: Nicht „Wie tippe ich SQL von Hand?", sondern „Wie prüfe ich, ob ein Modell korrekt in ein Schema übersetzt wurde?". Dieselbe Haltung kennen wir bereits aus `08b_CodeReview`: Anforderungen lesen, Artefakt danach prüfen, fehlende Bedingungen sichtbar machen.

DDL: Was wird hier festgelegt?

`DDL` steht für *Data Definition Language*. Mit ihr beschreiben wir nicht die einzelnen Datensätze, sondern die Struktur der Datenbank selbst: Welche Tabellen es gibt, welche Spalten darin vorkommen und welche Regeln beim Speichern gelten.

Aus dem ER-Modell wird hier also ein technisches Schema. Genau deshalb ist DDL keine reine Syntaxübung, sondern die Implementierung einer fachlichen Spezifikation.

Minimale Form von `CREATE TABLE`

Die Grundidee von `CREATE TABLE` ist einfach: Eine Tabelle bekommt einen Namen, darin werden Spalten mit Datentypen aufgelistet, und zusätzlich können Constraints ergänzt werden. Das ist die minimale technische Form, mit der aus einer fachlichen Entität eine reale Datenbanktabelle wird.

Beim Übertragen aus dem ER-Modell helfen drei Leitfragen:

- Welche Entität wird zur Tabelle?
- Welche Attribute werden zu Spalten?
- Welche Beziehungen müssen später über Fremdschlüssel sichtbar werden?

Datentypen in SQLite

SQLite kennt nur wenige Grundtypen wie `INTEGER`, `REAL`, `TEXT` und `BLOB`. Für Lehrbeispiele ist das ein Vorteil: Wir müssen uns nicht in vielen produktspezifischen Details verlieren, sondern können die fachliche Passung diskutieren.

Wichtig ist dabei: Ein Datentyp ist keine bloße Formalität. Er drückt aus, ob ein Attribut als Zahl, Text oder Binärwert gedacht ist. Wenn ein KI-Werkzeug hier einen unpassenden Typ vorschlägt, ist das kein Stilproblem, sondern ein fachlicher Fehler im Schema.

Beispiel: KI-generierte DDL

Gerade einfache Geometriebeispiele zeigen gut, warum Verifikation nötig ist. Eine KI kann für Punkte und Linien schnell formal korrektes SQL erzeugen. Das heißt aber noch nicht, dass wirklich alle Regeln des Modells umgesetzt wurden.

Man sollte deshalb nicht mit der Frage starten, ob das SQL „schön“ aussieht, sondern ob jede fachliche Aussage aus dem ER-Modell in der DDL wiederzufinden ist. Schon bei kleinen Beispielen können Pflichtattribute, Eindeutigkeit oder Beziehungsregeln fehlen.

DDL-Verifikationscheckliste I

Der erste Teil der Checkliste prüft, ob die Grundstruktur des Modells überhaupt vollständig übernommen wurde. Dazu gehört:

- jede Entität als eigene Tabelle,
- jedes Attribut als passende Spalte,
- fachlich plausible Datentypen,
- und ein definierter Primärschlüssel.

Wenn hier etwas fehlt, ist die DDL zwar oft noch ausführbar, aber sie bildet das Modell nicht mehr korrekt ab. Genau das macht KI-generierte SQL-Texte tückisch: Sie können technisch gültig und inhaltlich trotzdem falsch sein.

Gerade diese zweite Hälfte der Checkliste ist in der Praxis entscheidend: KI-Systeme erzeugen häufig „fast richtige“ Tabellen, lassen aber Zwangsbedingungen weg. Das Ergebnis funktioniert technisch, schützt aber die Datenqualität nicht. Deshalb ist das ER-Diagramm hier die Referenz.

Agentischer Workflow: ER → SQL → Review

Der sinnvolle Arbeitsablauf besteht aus drei Schritten. Zuerst steht das ER-Modell als Spezifikation. Danach kann ein KI-System daraus SQL erzeugen. Der entscheidende dritte Schritt ist aber die Review-Phase: Das erzeugte Schema wird Punkt für Punkt gegen das Modell geprüft.

Damit wird aus „Prompten“ eine kontrollierte Ingenieurarbeit. Die KI liefert einen Vorschlag, wir übernehmen die Verantwortung für die Verifikation. Wenn Constraints fehlen oder Beziehungen falsch übersetzt wurden, muss nachgebessert oder nachgepromptet werden.

Auch beim Einfügen bleibt die Verifikationsperspektive wichtig: Wenn ein `INSERT` unerwartet akzeptiert wird, obwohl fachlich ein Fehler vorliegt, fehlt häufig ein Constraint in der DDL. Wenn ein `INSERT` abgelehnt wird, lohnt sich die Rückfrage: Ist das Modell verletzt oder ist die DDL falsch formuliert?

Tabelle aus vorhandenen Daten erzeugen

Mit `CREATE TABLE ... AS SELECT ...` kann man schnell eine neue Tabelle aus bestehenden Daten ableiten. Das ist praktisch für Zwischenergebnisse, Materialisierung oder Kopien für weitere Analysen.

Didaktisch wichtig ist aber die Einschränkung: Solche Tabellen übernehmen die fachliche Absicherung oft nicht vollständig. Primärschlüssel, Fremdschlüssel, `NOT NULL` oder `UNIQUE` müssen deshalb erneut geprüft und gegebenenfalls explizit ergänzt werden.

SQL ist nicht universell: Netzwerke

Relationale Tabellen sind stark, wenn Daten klar in Zeilen, Spalten und Beziehungen organisiert werden können. Für Netzwerke wie Straßennetze, Leitungsnetze oder Erreichbarkeitsprobleme ist das jedoch nicht immer die natürlichste Form.

Solche Modelle bestehen oft primär aus Knoten und Kanten. Fragen wie Nachbarn, Wege oder kürzeste Routen lassen sich deshalb in Graphdatenbanken häufig direkter ausdrücken als in rein relationalen Tabellen.

SQL ist nicht universell: BIM und Dokumente

Auch BIM-Daten, semi-strukturierte Dokumente oder JSON-ähnliche Daten passen nicht immer gut in starre relationale Schemata. Viele Eigenschaften sind dort optional, verschachtelt oder ändern sich im Projektverlauf.

Dann kann eine Dokumentdatenbank flexibler sein als ein enges Tabellenschema. Die Grundregel lautet deshalb nicht „SQL immer zuerst“, sondern: Zuerst das Informationsmodell verstehen, dann den passenden Datenbanktyp auswählen.

Was Sie mitnehmen sollten

Das ER-Modell bleibt die Spezifikation, die DDL ist ihre technische Umsetzung. KI darf `CREATE TABLE` formulieren, aber die fachliche Verantwortung bleibt bei uns: Tabellen, Attribute, Datentypen und Constraints müssen systematisch gegen das Modell geprüft werden.

Wenn Constraints fehlen, entsteht nicht nur ein Schönheitsfehler, sondern ein echtes Datenqualitätsrisiko. Und schließlich gilt auch hier ein grundlegendes Ingenieurprinzip: Nicht jedes Informationsmodell gehört automatisch in eine relationale Datenbank.

Referenzen

Bibliographie